

Single-Metric Size and Complexity Thresholds Fail to Forecast Refactoring*

Binamra Aryal[✉]

University of Texas at Arlington

Arlington, USA

bx7604@mavs.uta.edu

Abstract

Code smells, like excessive size or complexity, surface as technical debt. Static analysers flag them when metrics cross fixed thresholds rather than learning from developer behaviour. We ask whether these thresholds match where developers refactor. Adopting a revealed-preference view, we mine five Java systems, track files across renames, and learn per-metric thresholds from past refactorings. We test these blind against tool defaults and a naive median under abundant, scarce, and forecasting scenarios. Results are consistently negative: no single size or complexity metric usefully separates refactored from untouched files, and even the best comes nowhere near usable precision. Re-tuning does not help: the calibrated threshold, the default, and a naive median all sit in the same low range. The default thresholds are miscalibrated, but the single-metric premise fails for size and complexity.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**; *Software evolution*.

Keywords: code smells, refactoring, technical debt, mining software repositories, static analysis, thresholds

ACM Reference Format:

Binamra Aryal. 2026. Single-Metric Size and Complexity Thresholds Fail to Forecast Refactoring. In *Companion Proceedings of the 2026 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3837729.3839569>

1 Problem and Motivation

Static analysers (PMD, Checkstyle [6], SonarQube [17]) flag a code smell when a source metric crosses a fixed threshold, e.g., a method whose cyclomatic complexity exceeds PMD’s default bound of 10 is “too complex”. A class swollen in size, methods, and public interface may be a “God Class” [11, 12].

*Replication package: <https://github.com/Binamra00/rs-replication>



This work is licensed under a Creative Commons Attribution 4.0 International License.

SPLASH Companion '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2910-2/2026/10

<https://doi.org/10.1145/3837729.3839569>

Crossing the threshold flags code worth restructuring before its technical debt [7] compounds. But thresholds are derived from metric distributions [2, 10, 13], not from developer behaviour. Work that does target behaviour predicts refactoring with multivariate models [3, 8, 9], leaving untested the single-metric premise analysers ship: metrics tie to refactoring only weakly [4, 5, 14–16], but no threshold has been tuned to a project’s own repairs. We therefore take a *revealed-preference* view and ask **(RQ)**: *Can any single standard size or complexity metric, with its threshold tuned to a project’s own past refactorings, forecast which files its developers refactor next?*

2 Methodology

A file bundles methods and inner classes, each with its own metric reading, so our unit is the code *entity*, a method or class. We draw metrics from the PMD Java Design rules built on a single tunable threshold and group them by the canonical catalogue [11, 12] into smells (Table 2). We keep the size and complexity smells, God Class and Long Method, and exclude Feature Envy, whose PMD metrics are class-level coupling counts, and Data Class (a boolean rule with no tunable metric). An observation is one metric’s value for one *entity* at a targeted release (Table 1), labelled *refactored* when a refactoring mapped to the metric’s smell targets that entity by the next parsed release.

Phase 1: Mining. Three miners feed the pipeline: PMD measures metrics at each targeted release snapshot; RefactoringMiner [1, 18, 19] detects refactorings (our ground truth) across all branches; and a metadata miner supplies commit SHAs and timestamps. From RefactoringMiner’s rename and move operations we build an *alias map* that threads each class and method’s identity across releases, so a renamed entity is not counted as new. Seeking a mature, multi-domain OSS baseline, we apply fixed inclusion criteria (AST-parseable, 0.15–1.5M analysable LOC, ≥20 parsed releases over a multi-year history, and architectural-domain diversity) to select five GitHub systems, relaxing Checkstyle’s parsed-release count (Table 1).

Phase 2: Data engineering. Each refactoring is attributed to the entity it targets, measured at the immediately preceding parsed snapshot. We form three scenarios per system from two splitting schemes. *Abundant* and *scarce* are

Table 1. Corpus, per-metric dataset, and forecasting result. *Releases/Targ.*: raw release tags, and the targeted subset — first and last of each minor-version era [20] (major for Checkstyle). *Unparsed*: targeted releases PMD could not analyse. *Zero-ref*: parsed releases with no mapped refactoring. *Rows*: metric observations, one per Table 2 metric per targeted release. *Refac.*: entities marked refactored by the next parsed release. *Aliases*: class rename/move links threading identity across releases. *BR*: base rate of the best-ranking metric (highest holdout AUC-PR). *AUC-PR*: that metric’s precision–recall area in the forecasting holdout; a no-skill ranker scores BR.

System	Years	Releases	Targ.	Unparsed	Zero-ref	Rows	Refac.	Files	Aliases	BR	AUC-PR
Commons-Lang	'02–'25	113	35	0	13	221K	2,340	230	551	0.6%	0.02
Checkstyle	'01–'26	211	26	12	2	81K	2,437	993	3,108	14.5%	0.10
Dubbo	'11–'26	158	24	0	4	505K	6,027	3,203	1,545	6.3%	0.07
JUnit	'17–'26	110	31	0	3	292K	1,991	780	1,435	1.6%	0.03
QuestDB	'14–'26	128	55	20	0	1.9M	15,294	4,378	5,071	1.0%	0.02

Table 2. Smell-to-refactoring taxonomy (revealed preference) [3, 11]. [†]absent from the dataset (no PMD metric).

Smell	PMD metrics	Repairing refactorings
God class	class NCSS, class CC, too many methods, too many fields [†] , excessive public count	extract class/interface/superclass/subclass, move class, move & rename class
Long method	method NCSS, method CC, excessive parameter list [†]	extract method, remove parameter

spatial splits, by file, so no file appears in both training and test, differing only in training-set size. *Forecasting*, the only scenario that forecasts, instead splits by date: it calibrates on the earlier releases (expanding-window cross-validation) and tests on the later releases.

Phase 3: Feature engineering. For each metric we scan for the threshold best separating refactored from untouched training entities. Because refactoring is rare, a flag-everything threshold can score well. A threshold is *identifiable* only if it beats both the base rate (Table 1) and flag-everything, two selection criteria (strict maximisation and a tolerance band) agree, and it holds across folds, excluding cherry-picked thresholds.

Phase 4: Evaluation. On held-out data, we compare the calibrated threshold against PMD’s default and a naive baseline, the training-set median. Independent of any threshold, we audit each metric by the area under its precision–recall curve (AUC-PR), an upper bound on any threshold’s separation.

3 Results

No single size or complexity metric usefully separates refactored files. In every system the metric that ranks refactoring best on the forecasting holdout (Table 1) is a class-level size or complexity metric. In the two systems whose holdout has enough positives to estimate discrimination reliably, that metric ranks refactoring no better than chance: Checkstyle’s

best AUC-PR (0.10) is below its 14.5% base rate, and Dubbo’s (0.07) barely exceeds its 6.3%. The other three have base rates near 1%, too few positives to estimate reliably.

Re-tuning does not forecast. In every scenario (abundant, scarce, and forecasting) and every system, not one metric clears the identifiability bar. Thresholds that score well in calibration collapse into the same low range as PMD’s default and the naive median: Commons-Lang class size falls from F_1 0.40 to 0.06, matching its default.

4 Conclusions

The answer to our RQ is no: a single standard size or complexity metric, thresholded on a project’s own past, cannot forecast which file developers refactor next. Refactoring reveals which files developers choose to repair; a single metric’s threshold flags candidate debt but not that choice.

5 Contributions and Future Work

This work contributes: (i) a rename-aware pipeline linking metrics to refactoring ground truth; (ii) a revealed-preference method separating real thresholds from artifacts, audited via AUC-PR; and (iii) evidence across five systems that single size and complexity thresholds fail to forecast refactoring.

Threats to validity. Findings are specific to PMD and five mature Java systems. PMD fails on several snapshots (Table 1), and snapshots without mapped refactorings lack positive labels. Checkstyle is sampled by major version, a cadence confound; metric and labelled refactoring come from adjacent snapshots, limiting the prediction horizon; the taxonomy labels a class relieved by extracted methods at method level, under-labelling large classes; and the alias map misses some renames. PMD lacks a faithful method-level coupling metric, so Feature Envy awaits future work.

Future work. We next ask not which file is smelly but which smelly file a developer *chooses* to repair, conditioning on triggers, ownership, and history, and whether multivariate models beat single ones [3, 8, 9].

References

- [1] Pouria Alikhanifard and Nikolaos Tsantalis. 2025. A Novel Refactoring and Semantic Aware Abstract Syntax Tree Differencing Tool and a Benchmark for Evaluating the Accuracy of Diff Tools. *ACM Transactions on Software Engineering and Methodology* 34, 2, Article 40 (2025), 63 pages. doi:10.1145/3696002
- [2] Tiago L Alves, Christiaan Ypma, and Joost Visser. 2010. Deriving metric thresholds from benchmark data. In *2010 IEEE International Conference on Software Maintenance*. IEEE, 1–10. doi:10.1109/ICSM.2010.5609747
- [3] Mauricio Aniche, Erick Maziero, Rafael Durelli, and Vinicius H. S. Durelli. 2022. The Effectiveness of Supervised Machine Learning Algorithms in Predicting Software Refactoring. *IEEE Transactions on Software Engineering* 48, 4 (2022), 1432–1450. doi:10.1109/TSE.2020.3021736
- [4] Gabriele Bavota, Andrea De Lucia, Massimiliano Di Penta, Rocco Oliveto, and Fabio Palomba. 2015. An experimental investigation on the innate relationship between quality and refactoring. *Journal of Systems and Software* 107 (2015), 1–14. doi:10.1016/j.jss.2015.05.024
- [5] Diego Cedrim, Alessandro Garcia, Melina Mongiovi, Rohit Gheyi, Leonardo Sousa, Rafael de Mello, Balduino Fonseca, Márcio Ribeiro, and Alexander Chávez. 2017. Understanding the impact of refactoring on smells: a longitudinal study of 23 software projects. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 465–475. doi:10.1145/3106237.3106259
- [6] Checkstyle Developers. 2024. Checkstyle: A development tool to help programmers write Java code that adheres to a coding standard. <https://checkstyle.sourceforge.io/>.
- [7] Ward Cunningham. 1992. The WyCash portfolio management system. In *ACM SIGPLAN OOPSLA'92 addendum to the proceedings*. 29–30. doi:10.1145/157709.157715
- [8] Dario Di Nucci, Fabio Palomba, Damian A Tamburri, Alexander Serebrenik, and Andrea De Lucia. 2018. Detecting code smells using machine learning techniques: are we there yet?. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 612–621. doi:10.1109/SANER.2018.8330266
- [9] Francesca Arcelli Fontana, Mika V Mäntylä, Marco Zanoni, and Alessandro Marino. 2016. Comparing and experimenting machine learning techniques for code smell detection. *Empirical Software Engineering* 21, 3 (2016), 1143–1191. doi:10.1007/s10664-015-9378-4
- [10] Matthieu Foucault, Marc Palyart, Jean-Rémy Falleri, and Xavier Blanc. 2014. Computing contextual metric thresholds. In *Proceedings of the 29th Annual ACM Symposium on Applied Computing (SAC)*. ACM, 1120–1125. doi:10.1145/2554850.2554997
- [11] Martin Fowler. 1999. *Refactoring: improving the design of existing code*. Addison-Wesley Professional.
- [12] Michele Lanza and Radu Marinescu. 2006. *Object-oriented metrics in practice: using software metrics to characterize, evaluate, and improve the design of object-oriented systems*. Springer Science & Business Media. doi:10.1007/3-540-39538-5
- [13] Paloma Oliveira, Marco Tulio Valente, and Fernando P. Lima. 2014. Extracting relative thresholds for source code metrics. In *2014 Software Evolution Week – IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering (CSMR-WCRE)*. IEEE, 254–263. doi:10.1109/CSMR-WCRE.2014.6747177
- [14] Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, and Andrea De Lucia. 2014. Do they really smell bad? A study on developers' perception of bad code smells. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 101–110. doi:10.1109/ICSM.2014.32
- [15] Jevgenija Pantiuchina, Michele Lanza, and Gabriele Bavota. 2018. Improving Code: The (Mis)Perception of Quality Metrics. In *2018 IEEE International Conference on Software Maintenance and Evolution (IC-SME)*. IEEE, 80–91. doi:10.1109/ICSM.2018.00017
- [16] Danilo Silva, Nikolaos Tsantalis, and Marco Tulio Valente. 2016. Why We Refactor? Confessions of GitHub Contributors. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. ACM, 858–870. doi:10.1145/2950290.2950305
- [17] SonarSource. 2024. SonarQube: Code Quality and Code Security. <https://www.sonarsource.com/products/sonarqube/>.
- [18] Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig. 2022. RefactoringMiner 2.0. *IEEE Transactions on Software Engineering* 48, 3 (2022), 930–950. doi:10.1109/TSE.2020.3007722
- [19] Nikolaos Tsantalis, Matin Mansouri, Laleh M Eshkevari, Davood Mazi-nanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering*. 483–494. doi:10.1145/3180155.3180206
- [20] Michele Tufano, Fabio Palomba, Gabriele Bavota, Rocco Oliveto, Massimiliano Di Penta, Andrea De Lucia, and Denys Poshyvanyk. 2015. When and why your code starts to smell bad. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 403–414. doi:10.1109/ICSE.2015.59

Received 2026-06-23; accepted 2026-07-25